QWEN

QWEN.md — ATMOSphere Lampa fork

Field	Value
Revision	1
Created	2026-05-23
Last modified	2026-05-23
Status	active
Status summary	Created per Phase 39.IT (User mandate 2026-05-23) — propagation of QWEN.md across the consumer fleet, mirroring CLAUDE.md + AGENTS.md per §11.4.35 canonical-root inheritance.
Issues	none
Continuation	—

INHERITED FROM constitution/QWEN.md

All rules in `constitution/QWEN.md` (and the `constitution/Constitution.md` it references) apply unconditionally. This module's rules below extend them — they do NOT weaken any universal clause. When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

The universal anti-bluff covenant (§11.4), no-guessing mandate (§11.4.6), credentials-handling mandate (§11.4.10), host-session safety (§12 + §12.6 + §12.10), and data safety (§9) all live in `constitution/Constitution.md`. Read it before working on any non-trivial change.

@constitution/QWEN.md

Canonical reference: https://github.com/HelixDevelopment/Helix-Constitution

How this file relates to CLAUDE.md + AGENTS.md

Per §11.4.35 canonical-root inheritance clarity:

- `constitution/QWEN.md` is the universal canonical root for the Qwen Code CLI.
- This file is the consumer-side extension for this submodule, carrying only the inheritance pointer + the §11.4 covenant anchors + a brief module summary.
- The full module ruleset lives in this submodule's sibling `CLAUDE.md`. Qwen Code agents MUST read CLAUDE.md before performing any work; this QWEN.md is the Qwen-specific entry point that ensures Qwen reads the inheritance pointer + anti-bluff covenant on every session.

Module summary

ATMOSphere's platform-signed fork of lampa-app/LAMPA — Kotlin Android client/wrapper for the Lampa media-library JS frontend. applicationId top.rootu.lampa kept identical to upstream.

For full module context (build steps, integration points, host-session safety, submodule-specific commit/push discipline) read this directory's `CLAUDE.md` and `AGENTS.md`.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28)

Forensic anchor — direct user mandate (verbatim):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end-user evidence; both are subject to the §8.1 five-constraint rule and §11 captured-evidence requirement.

Canonical authority: constitution submodule `Constitution.md` §11.4 (this end-user-quality-guarantee forensic anchor — propagation requirement enforced by pre-build gate `CM-COVENANT-114-QWEN-PROPAGATION`).

Non-compliance is a release blocker regardless of context.

Anti-bluff-manner clause (verbatim operator mandate — HRD-158 covenant cascade)

Forensic anchor — verbatim operator mandate:

> "IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

This clause is the leading verbatim operator anti-bluff mandate, restated here for cross-file consistency with this module's CLAUDE.md / AGENTS.md / CONSTITUTION.md. It does NOT weaken any inherited rule. Canonical authority: constitution submodule `Constitution.md` §11.4. Tests AND HelixQA Challenges are bound equally.

§11.4 extension anchors carried by this file

The following extension anchors apply unconditionally to every change landed in this submodule. Their full text lives in the sibling CLAUDE.md and in the canonical `constitution/Constitution.md`. Listed here so Qwen Code agents can locate them by literal string match:

- **§11.4.1 extension (Phase 33, 2026-05-05)** — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason and produces a FAIL exit code is just as misleading as a PASS-bluff. Fix at source layer, never at call sites.
- **§11.4.2 extension (Phase 34, 2026-05-06)** — Recorded-evidence requirement. Every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline.
- **§11.4.3 extension (Phase 34, 2026-05-06)** — Per-device-topology test dispatch. Topology-touching tests MUST detect topology at entry and dispatch the topology-appropriate variant.
- **§11.4.4 extension (User mandate, 2026-05-06)** — Test-interrupt-on-discovery + retest-from-clean-baseline. The moment any defect is re-discovered or newly identified mid-cycle, STOP and fix at root cause + four-layer coverage + full rebuild + reflash + retest.
- **§11.4.4 expansion (User mandate, 2026-05-06)** — Systematic debugging via superpowers skills + four-layer test coverage per fix (pre-build gate + post-build gate + post-flash on-device test + HelixQA Challenge + paired mutation) + documentation update + no-bluff certification per cycle.
- **§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)** — Audio: presence + channel count + sample rate + glitch census + coexistence-artifact census. Video: presence + routing target + frame health + obstruction census + resolution + codec. Challenges bound equally.
- **§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)** — Forbidden vocabulary: `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, `appears to`. Prove with captured evidence OR mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:`.
- **§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)** — Demotion from FAIL to lower-severity requires positive evidence captured under the same conditions that originally exposed the defect.
- **§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)** — Cite external source URL OR literal "NO external solution found — original work" in every non-trivial fix.
- **§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)** — All source-side fixes that DO

NOT require runtime on-device validation MUST be landed BEFORE the next firmware rebuild.

- **§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)** — Credentials NEVER live in tracked files. `.env` git-ignored project-wide. Tests load from `scripts/testing/secrets/` (chmod 600).
- **§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)** — When an HDMI sink with network-accessible introspection API is present, every audio test MUST consume the sink's report as captured-evidence.
- **§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)** — Every test that issues `am start` / `cmd media_session play` MUST issue matching `am force-stop` / `KEYCODE_MEDIA_STOP` + EXIT trap.
- **§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)** — Every active item in `Issues.md` carries a `**Status:**` line.
- **§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)** — Every active item in `Issues.md` carries a `**Type:**` line with one of `{Bug | Feature | Task}`.
- **§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)** — Release tag MUST NOT be created until a complete retest with ALL existing tests has been executed on a clean baseline.
- **§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)** — Every force-push MUST be preceded by a mechanical 4-step merge-first pipeline (fetch + integrate + audit + push with `--force-with-lease`).

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Project procedures MUST NOT use more than **60% of total system RAM** (`HOST_SAFETY_MAX_MEM_PCT`). The remaining 40% is reserved for the operator's other workloads. There is NO operator-facing override flag.

`scripts/build.sh` wraps `m -j` in `bounded_run` so only the bounded scope is OOM-killed if the build's collective RSS exceeds the budget — `user@<uid>.service` stays alive.

Canonical authority: constitution submodule `Constitution.md` §12.6.

Non-compliance is a release blocker regardless of context.

§11.4.101 — Autonomous-decision-over-blocking (User mandate, 2026-05-28)

Forensic: "when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"

In autonomous / endless-loop mode (per §11.4.87) the agent MUST minimize operator-blocking and make the safe, reliable, reversible decision itself — work NOT stalled overnight waiting for input. §11.4.87 = keep working; §11.4.101 = HOW to clear the decision points.

Decision rule (proceed autonomously when ALL): (a) reversible OR pre-op backup per §9.2; (b) safe choice determinable from captured evidence per §11.4.6 (`LIKELY` ≠ determination); (c) wrong-choice blast radius bounded AND recoverable; (d) composes with §11.4 + §12 + §9. Block-only-when (BLOCK via §11.4.66 ONLY when ALL): irreversible AND high-blast-radius AND choice undeterminable from evidence — external-account state, inaccessible hardware, destructive-op-without-backup, force-push (§9.2 + §11.4.41), spending / third-party send. `Operator-blocked` per §11.4.21 only after this fires + self-resolution-exhaustion audit. Maximize-progress-while-blocked: a block parks one work unit, not the loop — keep progressing NON-blocked items per §11.4.87 + §11.4.94; pose-and-idle = §11.4.94 + §11.4.97 violation. Composes §11.4.6/21/40/41/66/87/94/§9.2/§12. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-101-PROPAGATION` (literal `11.4.101`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.101.

Non-compliance is a release blocker regardless of context.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation. Acceptable autonomous paths: (a) programmatic instrumentation APK + structured JSON result; (b) headless intent dispatch + state poll (`am start --es` / `am broadcast` + `dumpsys` / `/proc/<pid>/maps` / `media.metrics`); (c) ADB-driven uiautomator (ONLY if hierarchy has ≥1 clickable node — empty hierarchy demands fallback to APK/intent); (d) network-side sink

probe per §11.4.13; (e) HelixQA autonomous QA session per §11.4.27. Coverage ledger (§11.4.25) classifies each feature `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`; `OPERATOR_ATTENDED_ONLY` blocks release until migrated. Composes §11.4.25/27/39/43/48/49/50/51. Pre-build gates `CM-COVENANT-114-52-PROPAGATION` + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE` + paired mutations. No escape hatch.

Canonical authority: constitution submodule `Constitution.md` §11.4.52.

Non-compliance is a release blocker regardless of context.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape. Helper contracts: `ab_pass_with_evidence <description> <evidence_path>` (verifies path exists AND non-empty); `ab_skip_with_reason <description> <closed-set-reason>` (`geo_restricted` / `operator_attended` / `hardware_not_present` / `topology_unsupported` / `network_unreachable_external` / `feature_disabled_by_config`); bare `ab_pass` deprecated (WARN pre-grace, FAIL post-grace 2026-06-19). Pre-build gates `CM-SINK-EVIDENCE-PER-FEATURE` + `CM-NO-FAIL-OPEN-SKIP` + `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` + `CM-COVENANT-114-69-PROPAGATION` + paired §1.1 mutations. Composes §11.4.1/2/5/6/13/27/50/52/68. No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§11.4.98 — Full-Automation Anti-Bluff Mandate — Live tests MUST be re-runnable end-to-end without manual intervention (User mandate, 2026-05-28)

Forensic: "Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun

endless times when needed! ... no false positives ... No bluff is allowed!" A live/integration/e2e/Challenge test requiring a human action during execution (typing a chat message, clicking a UI, hand-triggering a webhook, anything beyond test start + PASS/FAIL report) is by definition a §11.4 PASS-bluff at the automation layer. (A) Every test — unit/integration/e2e/Challenge/stress/chaos/live — MUST be fully self-driving end-to-end, reporting PASS/FAIL/SKIP-with-reason without further human action after startup. (B) Single exception: one-time credential bootstrap OUTSIDE test execution. (C) Live messenger/channel/agent tests: no operator-typed prompts (drive programmatically), no dev-session-colliding UUIDs, no 60s human-response windows (§11.4.50), re-runnability proof at `-count=3`, obsolescence audit COMPLIANT vs NON-COMPLIANT, no false-positive PASS. (D) Composes §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94. (E) Pre-build gate `CM-COVENANT-114-98-PROPAGATION` enforces literal `11.4.98` presence; paired §1.1 mutations strip → gate FAILs. (F) Manual-dependency test not rewritten within 30 days graduates to §11.4.90 Obsolete citing §11.4.98.

Canonical authority: constitution submodule `Constitution.md` §11.4.98.

Non-compliance is a release blocker regardless of context.

Companion documents

File	Role
`CLAUDE.md`	Full module ruleset (Claude Code primary context)
`AGENTS.md`	Cross-agent mirror (OpenCode, Cursor, Aider, generic AI tooling)
`QWEN.md` (this file)	Qwen Code CLI entry point
`../../../../constitution/Constitution.md`	Universal canonical rules
`../../../../constitution/QWEN.md`	Universal Qwen entry point